

TEKNİK ANALİZ DÖKÜMANI

IT Servis Yönetim Sistemi

Mimari, güvenlik, veri ve iş akışı tasarımı

Hazırlayan

Kurum

Doküman Türü

Ukbe Taha Şahinkaya

Sakarya Üniversitesi, Yazılım Mühendisliği

Teknik Analiz Dökümanı

İçindekiler

1. Giriş	4
1.1. Amaç ve Kapsam	4
1.2. Mimari Tasarım Hedefleri	4
2. Sistem Mimarisi	5
2.1. Mimari Yaklaşım	5
2.2. Konteyner Diyagramı ve Yönlendirme	5
2.3. Bileşenler ve Sorumluluklar	6
3. Teknoloji Yığını	7
4. Güvenlik ve Kimlik Mimarisi	8
4.1. Keycloak ve OpenLDAP Entegrasyonu	8
4.2. Kimlik Doğrulama Akışı	8
4.3. Rol Bazlı Erişim Denetimi (RBAC)	8
4.4. Dahili Uç Noktalar ve Hız Sınırlama	10
5. İş Süreçleri ve jBPM İş Akışı	11
5.1. Talep Durum Makinesi	11
5.2. BPMN Süreç Modeli ve Sinyaller	12
5.3. SLA Mekanizması	12
5.4. Dayanıklılık: Devre Kesici ve Geri Çağrı	13
6. Veritabanı Tasarımı ve Veri Sözlüğü	14
6.1. Yaklaşım ve Standartlar	14
6.2. Varlık-İlişki Diyagramı	15
6.3. Tablo Envanteri	15
6.4. Veri Sözlüğü (Seçili Tablolar)	16
7. Arka Uç Mimarisi (REST API)	18
7.1. Katmanlı Yapı	18
7.2. Hata Yönetimi ve API Sözleşmesi	18
7.3. Kritik API Uç Noktaları	18
8. Ön Uç ve Mobil İstemciler	20
9. Gözlemlenebilirlik	21
10. Dayanıklılık ve Performans	22
11. Test ve Kalite Güvencesi	23
12. Dağıtım ve CI/CD	24
13. Sonuç ve Değerlendirme	25

Şekil ve Tablo Listesi

Şekil 1. Konteyner mimarisi ve trafik akışı	5
Şekil 2. Kimlik doğrulama ve yetkilendirme akışı	8
Şekil 3. Eklemeli rol modeli ve yetki eksenleri	9
Şekil 4. Talep durum makinesi	11
Şekil 5. jBPM SLA süreç akışı	12
Şekil 6. SLA duraklatma / sürdürme zaman çizelgesi	13
Şekil 7. Varlık-ilişki (ER) diyagramı	15
Şekil 8. Katmanlı arka uç mimarisi	18
Şekil 9. Gözlemlenebilirlik veri hattı	21
Şekil 10. CI/CD hattı	24

1. Giriş

1.1. Amaç ve Kapsam

Bu doküman, IT Servis ve Destek Talebi Yönetim Sistemi'nin güncel teknik mimarisini, tasarım kararlarını ve sistem gereksinimlerini ayrıntılı biçimde ortaya koymak için hazırlanmıştır. Sistem, ölçeklenebilir, güvenli ve gözlemlenebilir bir kurumsal destek platformu olarak modern yazılım mühendisliği disiplinleri kullanılarak inşa edilmiştir.

Doküman; sistem mimarisini, teknoloji yığını, güvenlik ve kimlik modelini, iş akışı (jBPM) entegrasyonunu, veritabanı tasarımını, arka uç ve ön uç mimarilerini, gözlemlenebilirlik altyapısını, test stratejisini ve dağıtım yaklaşımını kapsar. Sistemin iş ve kullanıcı bakış açısıyla değerlendirilmesi için ayrı bir *Analiz Dökümanı* hazırlanmıştır.

1.2. Mimari Tasarım Hedefleri

Mimari, aşağıdaki temel hedefler etrafında tasarlanmıştır:

- **Net görev ayrımı:** API, yapay zekâ, kimlik, iş akışı, veri ve gözlemlenebilirlik; her biri bağımsız, ayrı dağıtılabilir servisler olarak çalışır.
- **Durumsuz ve yatay ölçeklenebilir uygulama servisleri:** Oturum durumu sunucuda tutulmaz; tüm durum PostgreSQL ve Redis'te saklanır.
- **Dışsallaştırılmış yapılandırma:** 12-faktör ilkeleri: ortam değişkenleri, Spring profilleri ve `application.yml`.
- **Uçtan uca gözlemlenebilirlik:** Log, iz (trace) ve metrik üretimi; tümü tek bir arka uçta birleşir.
- **Zarif bozulma:** İş akışı veya yapay zekâ servisi kesilse bile çekirdek API çalışmaya devam eder.

Polyglot Monorepo

Sistem; beş derlenebilir bileşenin (arka uç, llm-service, web ön ucu, mobil uygulama, iş akışı kjar'ı) ve paylaşılan altyapının tek bir depo (monorepo) içinde `docker-compose.yaml` ile orkestre edildiği, konteynerleştirilmiş, çok dilli (polyglot) bir yapıdır.

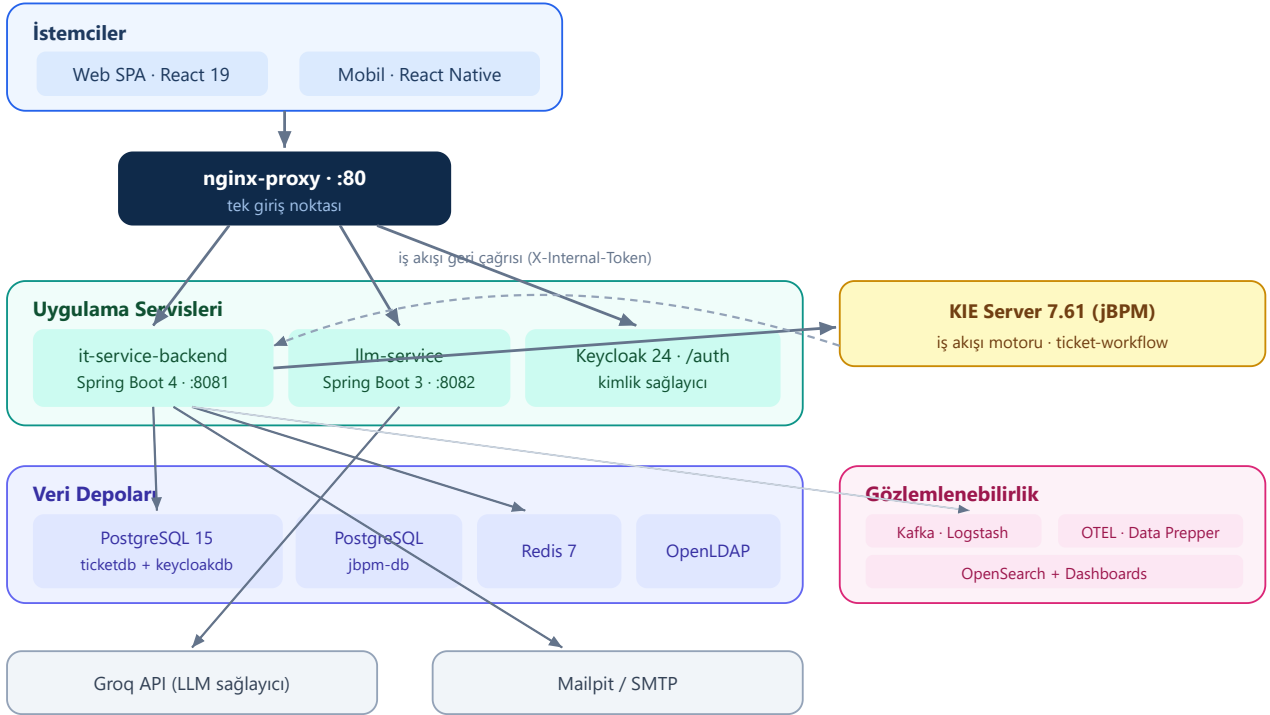
2. Sistem Mimarisi

2.1. Mimari Yaklaşım

Boyut	Yaklaşım
Topoloji	Mikroservise yakın: tek bir ters proxy (nginx) arkasında ayırık konteynerler
Arka uç	Klasik katmanlı mimari (Controller → Service → Repository)
Ön uç	Tek Sayfa Uygulaması (SPA, React) + React Native mobil istemci
İletişim	Senkron REST (JSON); canlı güncellemeler için STOMP/WebSocket; iş akışı motorundan HTTP geri çağrılar
Kimlik	Keycloak'a dışsallaştırılmış; böylece uygulamalar parola saklamaz
Durum	Uygulama servisleri durumsuzdur (JWT tabanlı); tüm durum PostgreSQL / Redis'te yaşar
Yapılandırma	12-faktör: ortam değişkenleri, Spring profilleri, <code>application.yml</code>
Şema yönetimi	Sürümlenmiş veritabanı geçişleri (Flyway); Hibernate <code>validate</code> modunda

2.2. Konteyner Diyagramı ve Yönlendirme

Tüm dış trafik, `80` numaralı port üzerinden **nginx** ters proxy'sine girer. Üretime yakın bir dağıtımda hiçbir uygulama servisi veya veri deposu doğrudan dışarıya açık değildir.



Oklar başlıca veri akışını gösterir; her bileşen yalnızca dahili köprü ağı üzerinden haberleşir.

Şekil 1. Konteyner mimarisi ve trafik akışı

nginx ters proxy'si gelen istekleri yola göre arka uç servislerine yönlendirir:

Yol	Hedef Servis
/	it-service-frontend (statik SPA)
/api/v1/	it-service-backend:8081
/api/v1/ai/	llm-service:8082
/auth/	keycloak-iam:8080

2.3. Bileşenler ve Sorumluluklar

Bileşen	Yığın	Sorumluluk
it-service-backend	Spring Boot 4 / Java 21	Talepler, SLA, kullanıcılar, yorumlar, ekler, bildirimler ve panolar için çekirdek REST API. ticketdb şemasının sahibi.
llm-service	Spring Boot 3 / Java 21	Groq API üzerinden yapay zekâ destekli talep özetleme. ticketdb 'yi izole bir Flyway geçmiş tablosuyla paylaşır.
it-service-frontend	React 19 + Vite	Role özel arayüzlere sahip web SPA; gezinme, kullanıcının rollerinin birleşiminden oluşturulur.
it-service-mobile	React Native + Expo	Web uygulamasıyla işlevsel eşdeğerlikte mobil istemci.
ticket-workflow-kjar	jBPM / BPMN 2.0	KIE Server'a dağıtılan ticket-lifecycle süreç tanımı.
Keycloak	Keycloak 24	Kimlik sağlayıcı: OAuth2/OIDC, TicketSystemRealm , LDAP'tan federe edilen kullanıcılar.
OpenLDAP	OpenLDAP	Dizin sunucusu; kullanıcı hesapları için doğruluk kaynağı.
KIE Server	jBPM 7.61 (WildFly)	İş akışı sürecini barındırır; ayrı bir PostgreSQL ile desteklenir.
PostgreSQL	PostgreSQL 15	ticketdb (uygulama) ve keycloakdb ; jbpm-db ayrı bir örnektir.
Redis	Redis 7	Dağıtık hız sınırı kovaları (Bucket4j); gelecekteki önbellek/kuyruk kullanımı için altyapı.
Kafka + Logstash	Kafka 3.7	Logları OpenSearch'e taşıyan tampon ve tüketici.
OTEL Collector + Data Prepper	OpenTelemetry	Telemetri alımı; izleri/logları/metrikleri OpenSearch'e dağıtır.
OpenSearch	OpenSearch 3.6	Loglar, izler ve metrikler için Dashboards ile birleşik depo.
nginx	nginx	Ters proxy ve tek giriş noktası.
data-generator	Java	API üzerinden gerçekçi demo verisi üreten bağımsız araç.

3. Teknoloji Yığını

Aşağıdaki tablolar, projede fiilen kullanılan teknolojileri ve sürümlerini katmanlara göre gruplanmış olarak listeler.

Arka Uç (Backend)

Dil / Çalışma Ortamı	Java 21
Çatı (Framework)	Spring Boot 4.0.5
Güvenlik	Spring Security · OAuth2 Resource Server
Veri erişimi	Spring Data JPA · Hibernate
Göç (Migration)	Flyway (V1–V38)
Önbellek	Caffeine
Hız sınırlama	Bucket4j (Lettuce/Redis)
İş akışı istemcisi	kie-server-client 7.61
Dayanıklılık	Resilience4j (devre kesici)
Telemetri	Micrometer Tracing · OpenTelemetry
Günlükleme	Log4j2 (Logback hariç tutuldu)
API dokümantasyonu	springdoc OpenAPI (Swagger UI)

Mobil

Çatı (Framework)	React Native 0.81 · Expo 54
Gezinme	React Navigation 7
Kimlik	expo-auth-session (OIDC PKCE)
Güvenli depolama	expo-secure-store
Gerçek zamanlı	@stomp/stompjs 7
Uluslararasılaştırma	i18next · react-i18next

Ön Uç (Web)

Kütüphane	React 19.2
Paketleyici	Vite 8
Stillendirme	Tailwind CSS 4
Yönlendirme	react-router-dom 7
HTTP	axios
Kimlik	keycloak-js 26
Gerçek zamanlı	@stomp/stompjs 7 (WebSocket)
Grafikler	Recharts 3
İkonlar	lucide-react
Uluslararasılaştırma	i18next · react-i18next
Test	Vitest · Testing Library · jsdom

Veri, Altyapı & Gözlemlenebilirlik

Veritabanı	PostgreSQL 15 (ticketdb, keycloakdb, jbpmm-db)
Önbellek/Kuyruk	Redis 7
Kimlik & Dizin	Keycloak 24 · OpenLDAP
İş akışı	jBPM 7.61 / KIE Server (WildFly)
Mesajlaşma	Kafka 3.7 · Logstash
Telemetri	OTEL Collector · Data Prepper
Arama/Pano	OpenSearch 3.6 + Dashboards
Konteynerleştirme	Docker Compose · Kubernetes (kind/kustomize)
CI/CD	GitHub Actions · Docker Hub
Kalite	JaCoCo · SonarQube · Testcontainers

4. Güvenlik ve Kimlik Mimarisi

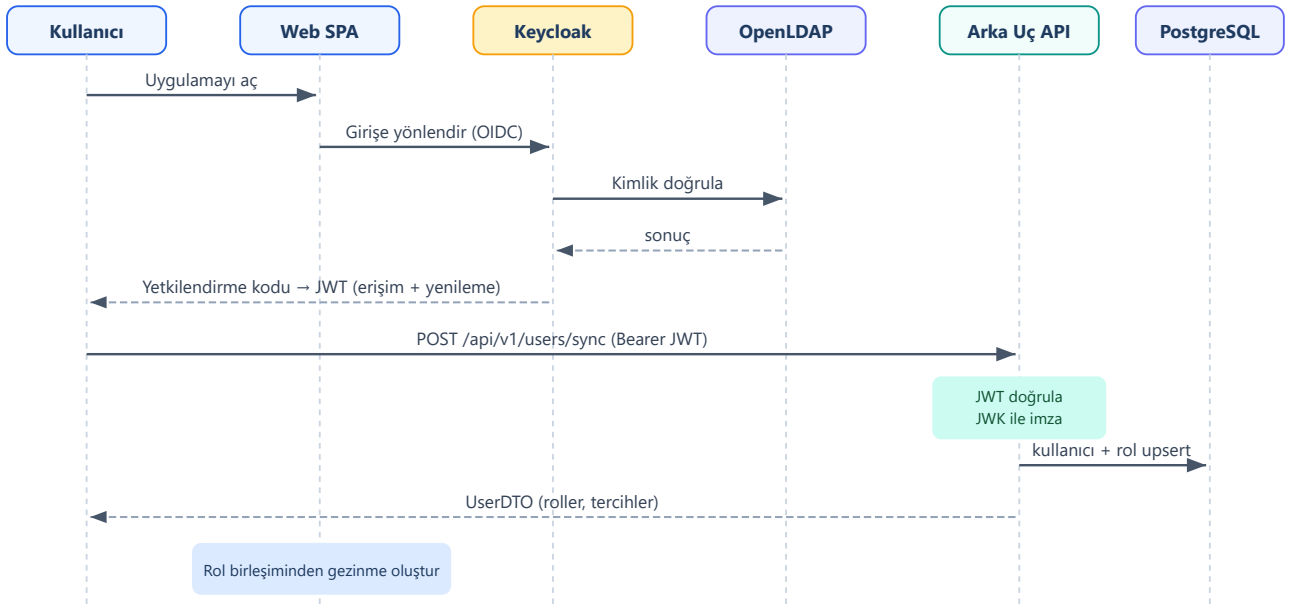
Güvenlik altyapısı, modern web uygulamalarında standart kabul edilen **durumsuz (stateless)** mimari üzerine kuruludur. Oturumlar sunucuda tutulmaz; güvenlik tamamen JWT (JSON Web Token) bazında uygulanır. Kimlik ve yetkilendirme, merkezî bir IAM aracı olan **Keycloak** ile bir izin sunucusu olan **OpenLDAP**'ın entegrasyonu aracılığıyla sağlanır.

4.1. Keycloak ve OpenLDAP Entegrasyonu

Kullanıcı kimlikleri merkezî olarak yönetilir ve dış sistemlerle senkronize edilir. Bu amaçla Keycloak ile OpenLDAP arasında **Kullanıcı Federasyonu (User Federation)** kurulmuştur:

- **OpenLDAP:** Kullanıcıların temel kimlik bilgilerini (kullanıcı adı, e-posta, ad, soyad) tutan doğruluk kaynağı. LDAP grupları, Keycloak realm rollerine eşlenir.
- **Keycloak (TicketSystemRealm):** Giriş isteklerini alır, parola doğrulamasını LDAP'a devreder ve başarılı bir girişte dijital kimlik görevi gören JWT'yi (RS256 ile imzalı) üretir.

4.2. Kimlik Doğrulama Akışı



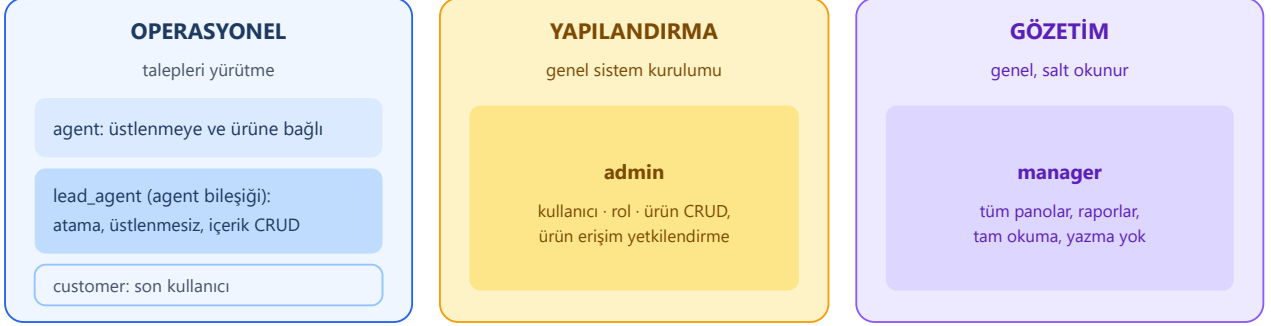
Şekil 2. Kimlik doğrulama ve yetkilendirme akışı

Arka uç saf bir **OAuth2 Resource Server**'dir: durumsuzdur, yalnızca JWT kabul eder ve imzayı anlık olarak Keycloak realm'inin JWK kümesine karşı doğrular. Sunucu tarafında oturum yoktur; CSRF devre dışıdır (çerez kullanılmaz).

4.3. Rol Bazlı Erişim Denetimi (RBAC)

Keycloak rolleri JWT içinde `realm_access.roles` altında gönderir. Özel bir `JwtAuthenticationConverter` bu rolleri `ROLE_*` yetkilerine dönüştürür. Uç nokta güvenliği, metot düzeyinde `@PreAuthorize` ek açıklamalarıyla uygulanır. Servis katmanındaki kapsam/üstlenme kararları `util/AuthRoles` yardımcılarından geçer.

Yetkilendirme modeli **eklemelidir**: bir kullanıcı bir rol kümesi taşır ve etkin hakları bu rollerin birleşimidir. Roller üç eksene yayılır. Tek kısıt, `customer` 'ın **tekil (singleton)** bir rol olmasıdır: `UserService` doğrulaması ve rol seçim pencereleri, `customer` 'ı herhangi bir personel rolüyle birleştirmeyi reddeder; böylece son kullanıcı ile personel kimlikleri ayırık tutulur.



Şekil 3. Eklemeli rol modeli ve yetki eksenleri

Aşağıdaki yetki matrisi, her rolün kritik modüllere erişimini özetler:

Rol	Talep Yönetimi	Yorum	Yapılandırma / Rapor
customer	Yalnızca kendi taleplerini görür ve oluşturur	Yalnızca EXTERNAL	Kişisel, kendine özel Genel Bakış panosu; CSAT doldurur
agent	Yetkili olduğu ürün havuzunu görür, üstlenir, durum değiştirir	EXTERNAL + INTERNAL	Kişisel Performansım panosu; efor kaydeder, SLA uyarıları alır
lead_agent	Atama yapar, üstlenmeden işlem yapar; ürün içeriğini yönetir	EXTERNAL + INTERNAL	Ürün kapsamlı ekip ve ürün bazlı panolar; temsilcilere derinlemesine iner
admin	Talep silme dâhil yönetsel işlemler	Tümü	Kullanıcı/rol/ürün CRUD + ürün erişim yetkilendirme; herhangi bir kullanıcının panosunu açar
manager	Tüm talepleri görür (salt okunur)	Tümünü görür	Tüm sistem, ürün bazlı ve kullanıcı bazlı panolar ve KPI raporları

Rol bilgileri Keycloak'ta tutulur, `user_roles` tablosunda (Flyway V37) önbelleğe alınır ve `/users/sync` ile senkronize edilir. Eski `users.role` sütunu yalnızca "birincil/görünen rol" olarak korunur; gerçek yetkilendirme JWT yetkileri + `@PreAuthorize` ile yapılır.

4.4. Dahili Uç Noktalar ve Hız Sınırlama

Konu	Uygulama
Dahili uç noktalar	<code>/api/v1/internal/**</code> JWT'yi atlar; paylaşılan bir <code>X-Internal-Token</code> başlığıyla korunur (yalnızca KIE Server geri çağırısı tarafından kullanılır). Karşılaştırma, zamanlama saldırılarına karşı sabit zamanlıdır.
Anonim izin listesi	Kimlik doğrulama uç noktaları, WebSocket el sıkışması, Swagger UI, <code>/actuator/health info metrics</code> .
Hız sınırlama	Bucket4j token-bucket, Redis üzerinden dağıtık; <code>application.yml</code> içinde yapılandırılan, temsilci başına genel bir sınır (<code>app.rate-limit.global-api.*</code> , <code>RATE_LIMIT_GLOBAL_*</code> ortam değişkenleriyle geçersiz kılınabilir).
2FA	Kullanıcı başına yapılandırılabilir TOTP (kimlik doğrulayıcı uygulama).
Girdi güvenliği	Tüm DTO'larda Bean Validation; ek olarak ek dosyalarında tür/boyut denetimleri ve metin dosyalarında hassas veri taraması.
Veri izolasyonu	Müşteri yalnızca kendi taleplerine erişir; temsilci yalnızca üstlendiği talepler / yetkili olduğu ürünlere; admin ve manager geneldir.

5. İş Süreçleri ve jBPM İş Akışı

Talep yaşam döngüsü, KIE Server üzerinde çalışan, yürütülebilir bir **BPMN 2.0 süreci** olarak modellenmiştir. Bir talep, doğası gereği uzun ömürlü, durumlu ve olay güdümlü bir varlıktır. Bu davranışı dağıtık Java koşulları yerine açık, görsel bir durum makinesiyle ifade etmek; izlenebilirlik, kalıcı bir SLA sayacı ve sinyal tabanlı geçişler sağlar.

5.1. Talep Durum Makinesi

BPMN, **tüm** talep durum değişiklikleri için yetkili durum makinesidir. Bir geçiş yalnızca arka uç `transition_<TARGET>` sinyalini gönderdiğinde ilerler; geçerli geçişler BPMN grafiğinin kendisinde kodlanmıştır. Geçersiz bir geçiş, motor tarafından sessizce yok sayılır ve servis katmanında HTTP `400` olarak ortaya çıkar.



Şekil 4. Talep durum makinesi

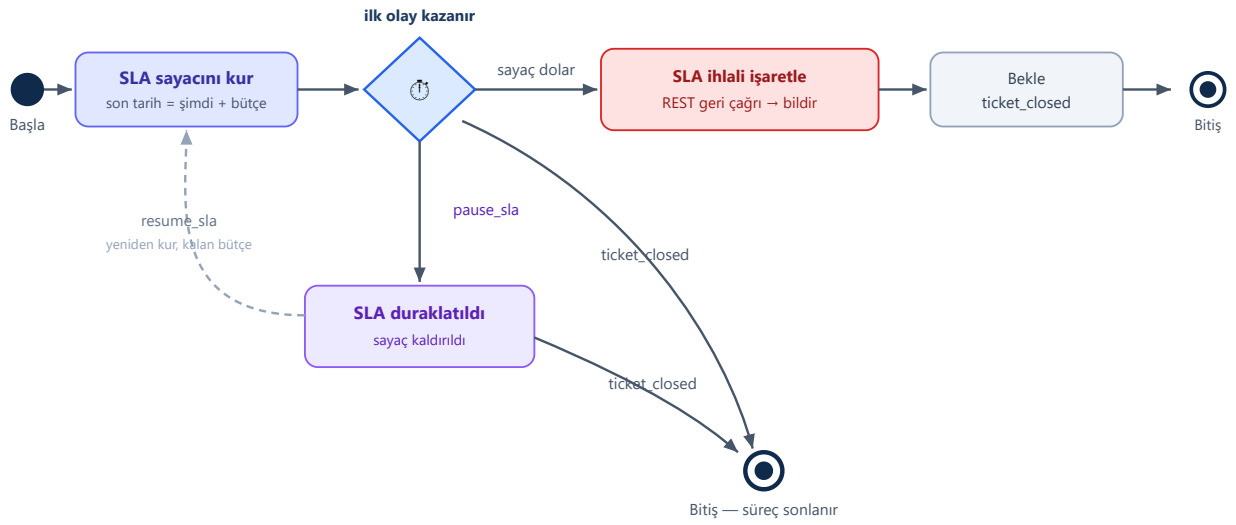
Kutular ayrıca SLA sayacının nerede **işlediğini** (**NEW**, **IN_PROGRESS**) ve nerede **duraklatıldığını** (**WAITING_FOR_CUSTOMER**, **RESOLVED**) gösterir. **DELETED** bir yaşam döngüsü durumu *değildir*: bir yöneticinin kalıcı silmesi, talebi veritabanından kalıcı olarak kaldıran, kayıt altına alınmış bir yan işlemidir; bu nedenle bir BPMN durum düğümü yoktur.

Yan etki geçişleri de BPMN'i sürer

Yalnızca kullanıcı tarafından başlatılan geçişler değil; *üstlenme/bırakma/atama* kaynaklı yan etki geçişleri de aynı `transition_<STATUS>` sinyalini gönderir. Böylece bir üstlenme `NEW → IN_PROGRESS` geçişini, son üstlenmenin bırakılması ise `IN_PROGRESS → NEW` geçişini gerçekleştirir; BPMN durum düğümü veritabanıyla tutarlı kalır.

5.2. BPMN Süreç Modeli ve Sinyaller

Süreç, aynı süreç örneğini paylaşan **iki paralel dal** çalıştırır: (1) talebin yetkili durum makinesi olan *durum dalı* ve (2) SLA sayacının etrafındaki olay tabanlı *SLA dalı*. SLA dalı, hangisi önce gerçekleşirse ona tepki verir: SLA sayacının dolması, `pause_sla` sinyali veya `ticket_closed` sinyali.



Durum makinesiyle (Şekil 4) paralel çalışır; CLOSED'a ulaşmak tüm dalları sonlandıran bir terminate-end olayı tetikler.

Şekil 5. jBPM SLA süreç akışı

Arka uç ile süreç arasındaki iletişim, adlandırılmış BPMN sinyalleriyle yapılır:

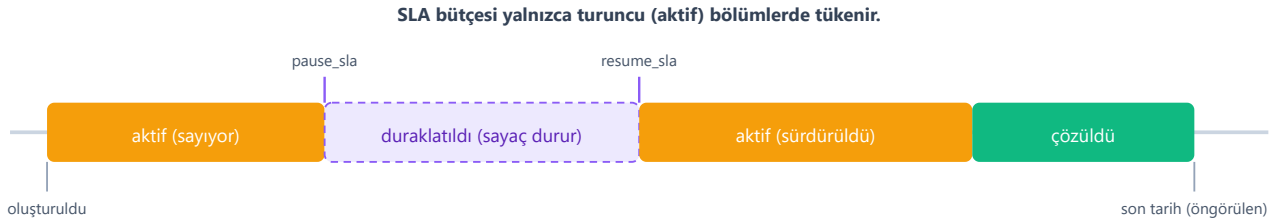
Sinyal	Gönderen (WorkflowService)	Amaç
<code>transition_<STATUS></code>	<code>requestStatusTransition()</code>	Durum düğümünü ilgili duruma ilerletir (NEW / IN_PROGRESS / WAITING_FOR_CUSTOMER / RESOLVED / CLOSED)
<code>pause_sla</code>	<code>pauseSla()</code>	SLA sayacını duraklatır; geçen süreyi <code>slaElapsedMs</code> 'ye ekler
<code>resume_sla</code>	<code>resumeSla()</code>	Kalan süreyi hesaplar ve sayacı yeniden kurar
<code>ticket_closed</code>	<code>closeTicketWorkflow()</code>	Süreci sonlandırır; sinyal hata verirse <code>abortProcess</code> 'e geri döner

5.3. SLA Mekanizması

SLA son tarihi, talep oluşturulduğunda önceliğe göre politikadan hesaplanır. Bir sayaç periyodik olarak çalışır, son tarihini geçen talepleri bir **ihlal** olarak işaretler ve bildirir; ayrıca uyarı eşiğine yaklaşanları işaretleyip atanmış temsilciye bildirir.

Öncelik	Hedef Çözüm	Uyarı Eşiği
CRITICAL	1 saat	son tarihte (0 sa)
HIGH	4 saat	1 saat önce
MEDIUM	24 saat	2 saat önce
LOW	72 saat	2 saat önce

SLA sayacı, talep `WAITING_FOR_CUSTOMER` veya `RESOLVED` durumundayken **duraklar** ve `IN_PROGRESS` 'e döndüğünde yeniden başlar. Yalnızca aktif (sayan) süre birikir (`slaElapsedMs`); sürdürmede son tarih, kalan aktif bütçe kadar ileri yansıtılır. Böylece müşteri kaynaklı gecikmeler ekibin aleyhine sayılmaz.



Şekil 6. SLA duraklatma / sürdürme zaman çizelgesi

5.4. Dayanıklılık: Devre Kesici ve Geri Çağrı

KIE Server, kritik olmayan bir bağımlılık olarak ele alınır: iş akışı motoru kullanılmadığı için talep işlemleri başarısız olmaz. Tüm KIE çağrıları bir Resilience4j **devre kesicisiyle** sarmalanmıştır. Sürecin durumu, kendine özel `jbpm-db` PostgreSQL örneğine kalıcı olarak yazılır (bellekte geçici H2 değil); böylece iş akışı örnekleri bir KIE Server yeniden başlatmasından sağ çıkar. Bir KIE `404` 'ü (örneğin BPMN örneği artık var olmayan, eskimiş bir `processInstanceId`) bir başarısızlık olarak ele alınmaz: devre kesiciyi tetiklemez ve arka uç veritabanı durum geçişini yine de kabul eder; böylece talep API'si tutarlı kalır.

Devre Kesici Ayarı	Değer
Hata oranı eşiği	%50
Kayan pencere boyutu	10 çağrı
Minimum çağrı sayısı	5
Açık durumda bekleme süresi	30 sn
Yarı açıkta izin verilen çağrı	3
Yavaş çağrı eşiği	10 sn / %50

KIE → Arka Uç Geri Çağrısı

SLA sayacı dolduğunda süreç, `POST /api/v1/internal/workflow/callback` uç noktasını çağırır. İstek `X-Internal-Token` ile doğrulanır; gövde `ticketId` ve `eventType` (`SLA_BREACHED` / `PROCESS_COMPLETED`) içerir. `SLA_BREACHED` işlenmesi **idempotent**'tir: bir ihlal zaten işaretlenmişse tekrar e-posta gönderilmez.

6. Veritabanı Tasarımı ve Veri Sözlüğü

6.1. Yaklaşım ve Standartlar

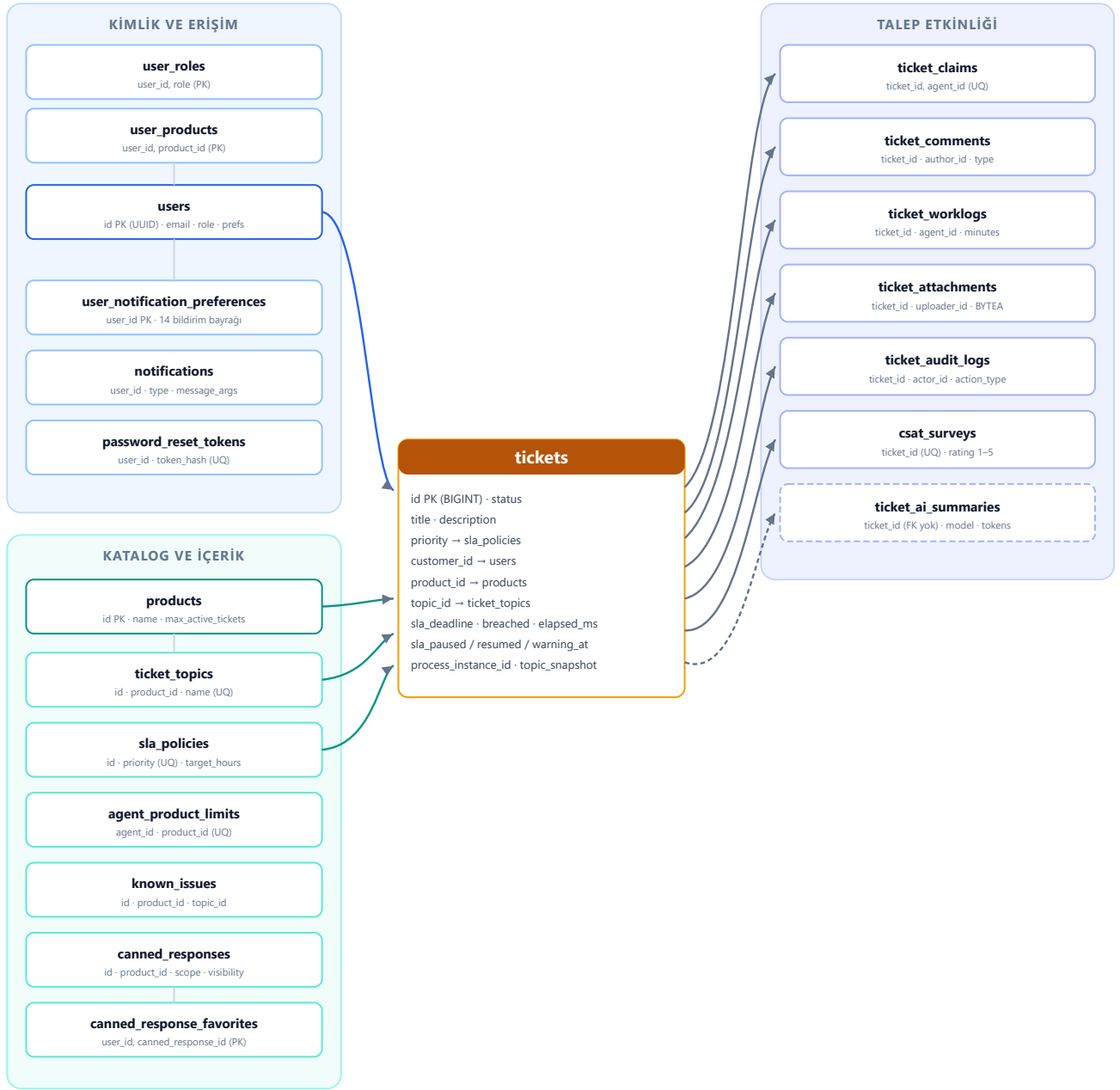
- **İlişkisel tasarım ve normalleştirme:** Şema, Üçüncü Normal Form'a (3NF) uygundur ve yabancı anahtar (FK) kısıtlarıyla güvence altına alınır.
- **Birincil anahtar stratejisi:** `users` tablosu Keycloak UUID'sini (VARCHAR(36)) kullanır; iş verileri (talepler, yorumlar vb.) performans için `BIGINT` kimlik kullanır.
- **Göç (Migration):** Tüm şema değişiklikleri sürümlenmiş Flyway betikleriyle (`V1 ... V37`) yönetilir. Hibernate `ddl-auto: validate` modundadır; şemayı asla değiştirmez.
- **Paylaşılan şema, izole geçmiş:** `llm-service` , `ticketdb` 'yi paylaşır ancak izole bir Flyway geçmiş tablosu (`flyway_schema_history_llm`) tutar.
- **Denetim ve bütünlük:** kritik tablolarda `created_at` / `updated_at` ; talebin alt tabloları silmede CASCADE ile bağlanır; denetim kayıtları yalnızca eklemelidir.

Platform üç mantıksal veritabanına yayılır; bunlardan yalnızca ilki uygulama tarafından tasarlanır ve sahiplenilir:

Veritabanı	Motor	Sahip ve içerik
<code>ticketdb</code>	PostgreSQL 15	Aşağıda ayrıntılandırılan uygulama şeması. Şemanın ve Flyway geçmişinin (<code>flyway_schema_history</code>) sahibi arka uçtur; llm-service tam da aynı veritabanını paylaşır , izole bir geçmiş tablosu (<code>flyway_schema_history_llm</code>) kullanır ve AI özetlerini <code>ticket_ai_summaries</code> 'ye yazar. Ayrı bir "LLM veritabanı" yoktur.
<code>keycloakdb</code>	PostgreSQL 15 (aynı örnek)	Keycloak'ın kendi kimlik şeması: kullanıcılar, oturumlar, istemciler ve realm yapılandırması. Tamamen Keycloak tarafından yönetilir ve uygulamaya kapalıdır.
<code>jbpm-db</code>	PostgreSQL (ayrı örnek)	KIE Server / jBPM süreç ve geçmiş deposu (süreç örnekleri, sayaçlar, denetim). İş akışı motoru tarafından yönetilir; devam eden SLA süreçlerini yeniden başlatmalar boyunca kalıcı tutar.

6.2–6.4 bölümleri yalnızca `ticketdb` uygulama şemasını modeller; Keycloak ve jBPM şemaları çatı tarafından yönetilir ve burada bilinçli olarak ayrıntılandırılmamıştır.

6.2. Varlık-İlişki Diyagramı



Şekil 7. Varlık-ilişki (ER) diyagramı (tüm uygulama tabloları)

6.3. Tablo Envanteri

Flyway V38 itibarıyla `ticketdb` uygulama şeması, aşağıdaki tabloları içerir (iki Flyway geçmiş tablosu hariç):

Tablo	Amaç
<code>users</code>	Keycloak ile senkronize kullanıcılar; dil/tema tercihleri
<code>user_roles</code>	Eklenebilir çok rollü RBAC için kullanıcı → rol eşlemesi (V37)
<code>user_products</code>	Kullanıcı ↔ yetkili ürün çok-açok bağlantısı
<code>products</code>	Destek ürünleri/kategorileri

Tablo	Amaç
ticket_topics	Ürün bazlı konu listesi
sla_policies	Önceliğe göre SLA hedefleri ve uyarı eşikleri
tickets	Ana destek talebi varlığı; SLA ve iş akışı alanları
ticket_claims	Temsilci ↔ talep üstlenmeleri (çoka-çok)
ticket_comments	Yorumlar (INTERNAL / EXTERNAL)
ticket_worklogs	Talep başına efor kayıtları (dakika)
ticket_attachments	Talep ekleri (BYTEA olarak saklanır)
ticket_audit_logs	Değiştirilemez (yalnızca eklenir) işlem/durum denetim kaydı
ticket_ai_summaries	İlm-service tarafından yazılan, önbellege alınmış AI üretimi talep özetleri (model, token sayıları, metin)
csat_surveys	Memnuniyet anketleri (1–5 puan)
notifications	Uygulama içi bildirimler (i18n anahtarı + argümanlar)
user_notification_preferences	Kullanıcı başına e-posta/uygulama içi bildirim tercihleri
known_issues	Ürün/konu bazlı bilinen sorunlar (bilgi tabanı)
canned_responses	Yeniden kullanılabilir hazır yanıt şablonları (kişisel/paylaşılan, çift dilli TR/EN)
canned_response_favorites	Kullanıcı başına favori hazır yanıtlar (bağlantı tablosu)
agent_product_limits	Temsilci-ürün başına aktif talep sınırı
password_reset_tokens	Tek kullanımlık, karması alınmış parola sıfırlama jetonları

6.4. Veri Sözlüğü (Seçili Tablolar)

tickets sistemin ana tablosudur.

Sütun	Tür	Kısıt	Açıklama
id	BIGINT	PK	Otomatik artan talep numarası
title	VARCHAR(500)	NOT NULL	Talep başlığı
description	TEXT	NOT NULL	Sorun açıklaması
status	VARCHAR(30)	NOT NULL	NEW / IN_PROGRESS / WAITING_FOR_CUSTOMER / RESOLVED / CLOSED
priority	VARCHAR(10)	FK → sla_policies	LOW / MEDIUM / HIGH / CRITICAL
customer_id	VARCHAR(36)	FK → users	Talebi oluşturan müşteri
product_id	BIGINT	FK → products	İlgili ürün (boş bırakılabilir)
topic_id	BIGINT	FK → ticket_topics	Konu (boş bırakılabilir, ON DELETE RESTRICT)

Sütun	Tür	Kısıt	Açıklama
topic_name_snapshot	VARCHAR(255)	boş bırakılabilir	Talep üzerinde yakalanan, normalsizleştirilmiş konu adı
sla_deadline	TIMESTAMP TZ	boş bırakılabilir	Hesaplanan hedef çözüm tarihi
sla_breached	BOOLEAN	DEFAULT false	SLA ihlal bayrağı
sla_elapsed_ms	BIGINT	DEFAULT 0	Biriken aktif SLA süresi
sla_paused_at / sla_resumed_at	TIMESTAMP TZ	boş bırakılabilir	SLA sayacının en son duraklatıldığı / sürdürüldüğü an
sla_warning_sent_at	TIMESTAMP TZ	boş bırakılabilir	Yaklaşan son tarih uyarısının gönderildiği an (yinelemeyi önler)
process_instance_id	BIGINT	boş bırakılabilir	jBPM süreç örneği tanımlayıcısı
created_at / resolved_at / closed_at	TIMESTAMP TZ	—	Yaşam döngüsü zaman damgaları

ticket_comments

Sütun	Tür/Kısıt
id	BIGINT PK
ticket_id	FK → tickets (CASCADE)
author_id	FK → users
message	TEXT (≤ 500 karakter)
type	EXTERNAL / INTERNAL
created_at	TIMESTAMP TZ

csat_surveys

Sütun	Tür/Kısıt
id	BIGINT PK
ticket_id	FK → tickets · UNIQUE
rating	INTEGER (1–5)
comment	TEXT (boş bırakılabilir)
created_at	TIMESTAMP TZ

ticket_ai_summaries (llm-service tarafından yazılır)

Sütun	Tür/Kısıt
id	BIGINT PK
ticket_id	BIGINT (özetlenen talep)
model	VARCHAR(100)
prompt/completion_tokens	INTEGER (boş bırakılabilir)
summary	TEXT
created_at	TIMESTAMP TZ

canned_responses

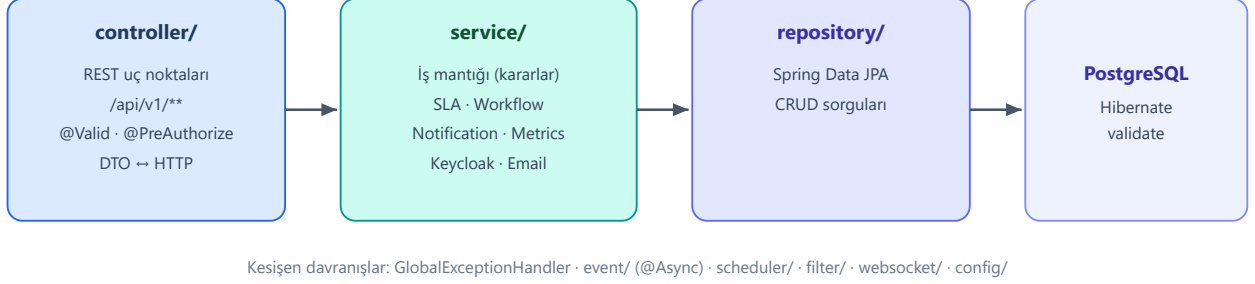
Sütun	Tür/Kısıt
id	BIGINT PK
title · shortcut	VARCHAR
content_tr / content_en	TEXT (≥ 1 zorunlu)
scope	PERSONAL / SHARED
visibility	EXTERNAL / INTERNAL / BOTH
owner_agent_id · product_id	sahip · ürün (boş bırakılabilir)

Enum değer kümeleri: Talep durumu (5), öncelik (LOW/MEDIUM/HIGH/CRITICAL), yorum türü (INTERNAL/EXTERNAL), hazır yanıt kapsamı (PERSONAL/SHARED) ve görünürlüğü (EXTERNAL/INTERNAL/BOTH), bildirim türü (TICKET_CREATED, TICKET_ASSIGNED, TICKET_STATUS_CHANGED, COMMENT_ADDED, SLA_WARNING, SLA_BREACHED, TICKET_RESOLVED, GENERAL), roller (customer/agent/lead_agent/admin/manager).

7. Arka Uç Mimarisi (REST API)

7.1. Katmanlı Yapı

Sunucu tarafı, "Görev Ayrımı" (Separation of Concerns) ilkesine dayalı, bakımı ve test edilmesi kolay bir katmanlı mimaridir.



Şekil 8. Katmanlı arka uç mimarisi

Paketler: `controller/`, `service/`, `repository/`, `entity/`, `dto/`, `event/` (`@EventListener` / `@Async`), `scheduler/` (cron), `filter/`, `interceptor/`, `websocket/` (STOMP), `config/` ve `exception/`. DTO'lar API sınırını oluşturur; JPA varlıkları istemciye asla doğrudan serileştirilmez.

7.2. Hata Yönetimi ve API Sözleşmesi

Uygulama genelindeki hatalar (talep bulunamadı, yetkisiz erişim, doğrulama hatası vb.) `@RestControllerAdvice` ile merkezî olarak yakalanır. Kullanıcı, ham bir Java yığın izi yerine standartlaştırılmış bir JSON hata zarfı alır. Bean Validation mesajları yerelleştirilmiştir (Türkçe/İngilizce). API, springdoc OpenAPI ile belgelenir ve Swagger UI üzerinden canlı test edilebilir.

7.3. Kritik API Uç Noktaları

Tüm uç noktalar `/api/v1/...` ön ekiyle sunulur. Temsili bir alt küme aşağıda listelenmiştir (gerekli rol `@PreAuthorize` ile uygulanır):

Metot	Uç Nokta	Açıklama	Rol
POST	<code>/api/v1/tickets</code>	Yeni bir talep oluşturur	customer
GET	<code>/api/v1/tickets/pool</code>	Atanmamış (NEW) talepleri listeler	agent, lead_agent
GET	<code>/api/v1/tickets/{id}</code>	Talep detayını getirir	kimliği doğrulanmış
PUT	<code>/api/v1/tickets/{id}/claim</code>	Talebi üstlenir (NEW→IN_PROGRESS)	agent, lead_agent
PUT	<code>/api/v1/tickets/{id}/assign</code>	Talebi başka bir temsilciye atar	lead_agent, admin
PUT	<code>/api/v1/tickets/{id}/status</code>	Durumu değiştirir (BPMN doğrular)	customer*, agent, lead_agent
PUT	<code>/api/v1/tickets/{id}/close</code>	Talebi bir gerekçeyle kapatır	agent, lead_agent
POST	<code>/api/v1/tickets/{id}/comments</code>	Bir yorum ekler (INT/EXT)	kimliği doğrulanmış

Metot	Uç Nokta	Açıklama	Rol
POST	<code>/api/v1/tickets/{id}/attachments</code>	Dosya yükler (≤10MB, denetlenir)	customer, agent, lead_agent
POST	<code>/api/v1/tickets/{id}/worklogs</code>	Bir efor kaydı ekler	agent, lead_agent
POST	<code>/api/v1/tickets/{id}/csat</code>	Memnuniyet anketini gönderir → kapanış	customer
GET	<code>/api/v1/tickets/{id}/sla-timer</code>	Canlı SLA geri sayımını getirir	customer, agent, lead_agent, manager
GET	<code>/api/v1/metrics/dashboard-summary</code>	Sistem/ekip panosu KPI'ları (tarih aralığı kapsamlı)	manager, lead_agent, admin
GET	<code>/api/v1/metrics/me/customer</code>	Çağıranın kendi müşteri Genel Bakış panosu	kimliği doğrulanmış
GET	<code>/api/v1/metrics/me/agent</code>	Çağıranın kendi temsilci Performansım panosu	agent, lead_agent
GET	<code>/api/v1/metrics/products/{id}/dashboard</code>	Ürün bazlı pano (lider ürün kapsamlıdır)	manager, lead_agent, admin
GET	<code>/api/v1/metrics/users/{id}/agent</code>	Gözetim: bir kullanıcının temsilci panosunu görüntüler	manager, lead_agent, admin
POST	<code>/api/v1/users/sync</code>	Yerel kullanıcıyı JWT'den senkronize eder	kimliği doğrulanmış
POST	<code>/api/v1/users/admin/create</code>	Keycloak'ta bir kullanıcı oluşturur	admin
POST	<code>/api/v1/ai/summaries/.../generate</code>	Bir AI özeti üretir (llm-service)	dahili jeton
POST	<code>/api/v1/internal/workflow/callback</code>	jBPM olay geri çağırısı	dahili jeton

* Bir müşteri kendi talebinde yalnızca sınırlı durum geçişleri yapabilir (örneğin yanıt verdiğinde `WAITING_FOR_CUSTOMER` → `IN_PROGRESS`).

8. Ön Uç ve Mobil İstemciler

React & Vite Yapılandırması

Arayüz React 19 ve Vite 8 ile inşa edilmiştir. Stillendirme için Tailwind CSS 4 kullanılır; ikonlar `lucide-react`, grafikler `Recharts` tarafından sağlanır. API çağrıları, Keycloak bearer jetonunu ekleyen bir axios interceptor üzerinden yapılır.

Kimlik ve Oturum (keycloak-js)

Kimlik yönetimi `keycloak-js` ile yapılır; JWT bellekte tutulur ve her istekte `Authorization: Bearer` başlığına eklenir. Oturum durumu, rol kümesini yetenek yardımcısıyla (`isAdmin`, `isLeadAgent`, `hasAnyRole` vb.) birlikte sunan `AuthContext` 'te tutulur.

Dinamik Gezinme ve Rota Koruması

Kenar çubuğu, kullanıcının rollerinin **birleşiminden** oluşturulur. `ProtectedRoute`, izin verilen rolleri rol kümesine karşı kontrol eder; rolü olmayan kullanıcı bir döngüye girmek yerine bilgilendirici bir sayfaya yönlendirilir.

Anlık Bildirimler

STOMP/WebSocket (`@stomp/stompjs`) bağlantısı `AuthContext` üzerinden kurulur; talep detay olayları ve bildirimler bağlı istemcilere gerçek zamanlı iletilir.

Uluslararasılaştırma ve Tema

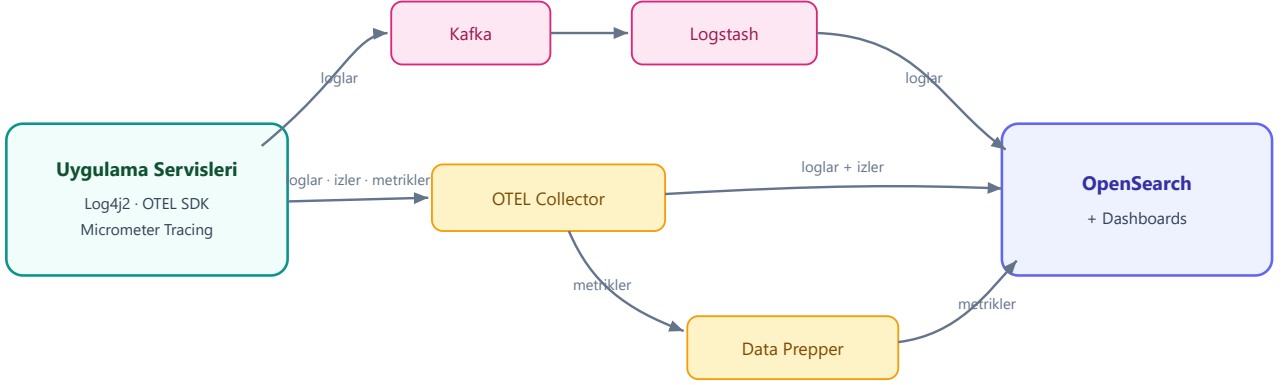
i18next ile Türkçe/İngilizce; tek ad alanlı dil dosyaları. Açık/koyu tema, SPA ile özel Keycloak giriş teması arasında alan kapsamlı bir çerez aracılığıyla paylaşılır.

Mobil Uygulama (React Native + Expo)

Web ile işlevsel eşdeğerlik; React Navigation 7 ile gezinme, OIDC PKCE girişi (`expo-auth-session`), güvenli jeton depolama (`expo-secure-store`) ve aynı STOMP tabanlı anlık bildirimler. Rol modeli birebir yansıtılmıştır.

9. Gözlemlenebilirlik

Platform **loglar**, **izler (traces)** ve **metrikler** üretir; bunların tümü OpenSearch'te birleşir. Tek bir gözlemlenebilirlik arka ucu, işletilmesi gereken hareketli parça sayısını azaltır.



Metrikler Data Prepper saptasını izler; çünkü Collector'ın OpenSearch dışı aktarıcısı metrik yazamaz (delta temporality).

Şekil 9. Gözlemlenebilirlik veri hattı

- **Loglar:** Log4j2 yapılandırılmış JSON üretir; hem Kafka → Logstash hem de OpenTelemetry log appender'ı üzerinden taşınır.
- **İzler (Traces):** Micrometer Tracing, OpenTelemetry'ye köprülenir; OTLP üzerinden dışa aktarılır (geliştirmede örnekleme 1.0).
- **Metrikler:** **Delta temporality** ile dışa aktarılır ve Data Prepper üzerinden yönlendirilir; çünkü collector'ın OpenSearch dışı aktarıcısı metrikleri işlemez. Delta temporality, OpenSearch `sum` toplamalarının `rate()` olmadan çalışmasını sağlar.

10. Dayanıklılık ve Performans

Mekanizma	Amaç
Devre kesici (Resilience4j)	jBPM/KIE Server hatalarını çekirdek API'den yalıtır; eskimiş bir süreç örneğini (KIE 404) tetiklenmeden tolere eder
Kalıcı iş akışı durumu	KIE süreç durumu <code>jbpm-db</code> PostgreSQL örneğine kalıcı yazılır; KIE Server yeniden başlatmalarından sağ çıkar
Önbellekleme (Caffeine)	Pano toplamaları 5 dakikalık TTL ile önbelleğe alınır; ilgili yazmalarda geçersiz kılınır
Hız sınırlama (Bucket4j + Redis)	API'yi genel olarak korur ve maliyetli AI çağrılarını kısıtlar
Bağlantı havuzu (HikariCP)	Sınırlandırılmış, ayarlanmış veritabanı bağlantı havuzu
Durumsuz servisler	Yapışkan oturumlar olmadan yatay ölçeklemeyi mümkün kılar
Zarif bozulma	İş akışı/AI kesintileri, sistemi çökertmeden işlevselliği azaltır
Asenkron işleme	Bildirim ve e-posta işleri <code>@Async</code> ile istek iş parçacığından uzak tutulur

İşlevsel olmayan hedef, normal yük altında tipik işlemler için ~2 saniyenin altında bir yanıt süresidir.

11. Test ve Kalite Güvencesi

Birim Testleri

Arka uç iş mantığı (servis katmanı) JUnit 5 ve Mockito ile test edilir; veritabanı ve dış API'ler taklit edilir (mock). Ön uç bileşenleri Vitest + Testing Library + jsdom ile doğrulanır.

Entegrasyon Testleri

Failsafe, `*IT.java` sınıflarını çalıştırır; **Testcontainers** ile gerçek bir PostgreSQL örneği (Redis dâhil) ayağa kaldırılır. API güvenliği ve RBAC senaryoları uçtan uca doğrulanır.

Kapsam ve Statik Analiz

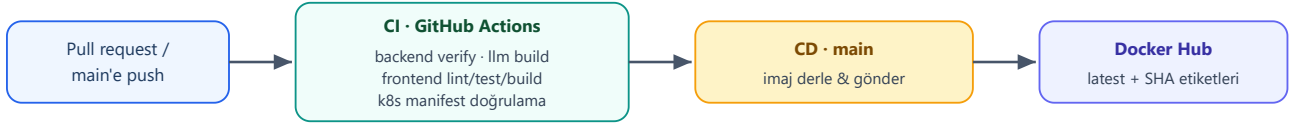
`mvnw verify`, JaCoCo kapsam raporunu üretir; statik kod analizi SonarQube ile yapılır. Ön uç için ESLint çalıştırılır.

Sürekli Entegrasyon

Her pull request ve `main` push'unda CI; arka uç `verify`, ILM-service derlemesi, ön uç lint+test+build ve Kubernetes manifest doğrulamasını çalıştırır.

12. Dağıtım ve CI/CD

Aynı imajlardan iki orkestrasyon yolu desteklenir: geliştirme ve demolar için **Docker Compose** (`docker-compose.yaml`) ve üretime yakın senaryolar için **Kubernetes** (`k8s/`, Kustomize). Kubernetes; ortam bağımsız bir `base/`, yerel kind kümesi için bir `overlays/local` ve HPA + cert-manager + SealedSecrets ekleyen bir `overlays/prod` içerir.



Şekil 10. CI/CD hattı

`main` üzerinde CI başarıyla tamamlandığında CD devreye girer: Docker imajları derlenir ve Docker Hub'a gönderilir; geri alma için hem `latest` hem de commit SHA'sı ile etiketlenir.

13. Sonuç ve Değerlendirme

Bu dokümanın kapsamında, modern bir IT Servis ve Destek Talebi Yönetim Sistemi'nin gerektirdiği tüm mimari katmanlar ayrıntılandırılmıştır. Aşağıdaki tablo, işlevsel olmayan gereksinimlerin nasıl karşılandığını özetler.

Gereksinim	Nasıl Karşılanır
Konteynerleştirme	Her bileşen izole bir konteynerdir; Docker Compose ve Kubernetes orkestrasyonu
Katmanlı arka uç	Controller → Service → Repository; dışsallaştırılmış yapılandırma; standart hata biçimi
Kalıcılık	Normalleştirilmiş PostgreSQL şeması, JPA/Hibernate, Flyway geçişleri
Güvenlik	Keycloak + LDAP, eklemeli RBAC, 2FA, OAuth2/OIDC; web ve mobil
İş akışı	Bir jBPM BPMN süreci, talep durum geçişlerini sürer
Gözlemlenebilirlik	OpenTelemetry izleri + metrikleri; OpenSearch'te birleşik log/iz/metrik panoları
Performans	Önbellekleme, havuzlama, hız sınırlama; ~2 sn altı yanıt hedefi
Test	JUnit 5/Mockito, Testcontainers entegrasyonu, JaCoCo, SonarQube
DevOps	GitHub Actions CI/CD; imaj derleme ve yayımlama
Dokümantasyon	Bu doküman, README, RUNBOOK ve etkileşimli OpenAPI/Swagger referansı

13.1. Temel Tasarım Kararları

Karar	Gerekçe
Dışsallaştırılmış kimlik (Keycloak + LDAP)	Standartlara dayalı SSO, 2FA ve federasyon; uygulamalar parolasız kalır
Durumsuz JWT resource server	Yatay ölçeklenebilirlik ve kimlik sağlayıcı ile API arasında temiz bir ayrım
Yaşam döngüsü için jBPM	Dağınık if/else durum mantığı yerine açık, gözden geçirilebilir bir BPMN süreci
İş akışı etrafında devre kesici	KIE Server çökse bile çekirdek talep API'si erişilebilir kalır
Tek bir gözlemlenebilirlik arka ucu	Loglar/izler/metrikler tek bir depoyu ve tek bir sorgu arayüzünü paylaşır
Flyway + <code>validate</code>	Şema sürümlenir ve gözden geçirilebilir; Hibernate onu sessizce değiştiremez
Paylaşılan <code>ticketdb</code> , izole geçmiş	AI servisi, servisler arası çağrı olmadan alan verisini kullanır